

Session Lecture

Attention Mechanism

The Foundation of Modern NLP
& Large Language Models

Made by Nancy Ahmed

Motivation — The Problem

? How can a model translate a sentence with 50 words if it compresses everything into **ONE** fixed vector?

ENCODER

Word 1

Word 2

Word 3

Word 4

Word 5

ONE
VECTOR

DECODER

Translation
Output

- ✗ Forgets early words
- ✗ Fails on long sentences
- ✗ Information bottleneck

The Coreference Challenge

What does "it" refer to? A good model must understand context.

The animal didn't cross the street because it was too tired.

✓ it → animal (tired animal, not the street)

The trophy doesn't fit in the suitcase because it is too big.

✓ it → trophy (big trophy, not the suitcase)

The trophy doesn't fit in the suitcase because it is too small.

✓ it → suitcase (suitcase is too small)

What is Attention?

Imagine reading a book. When answering a question, you don't re-read every page equally. You focus only on the relevant parts. Humans use attention — and neural networks can too.

Sentence:

Ahmed likes eating green apples.

Question: "Who likes apples?"

Attention weights:



Attention Intuition

Sentence

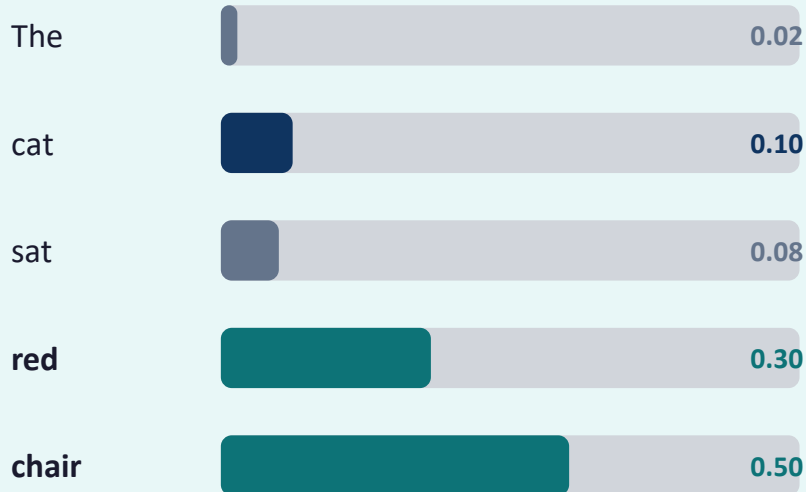
The cat sat on the red chair.

Task: Predict next word after "chair"

Not every word matters equally!

**Attention learns WHERE to look,
not everywhere equally.**

Attention Weights



Query, Key & Value

 Think of Google Search: you type a Query → Google compares it against Keywords (Keys) of billions of pages → Returns the most relevant content (Values).

Q
Query

What am I looking for?

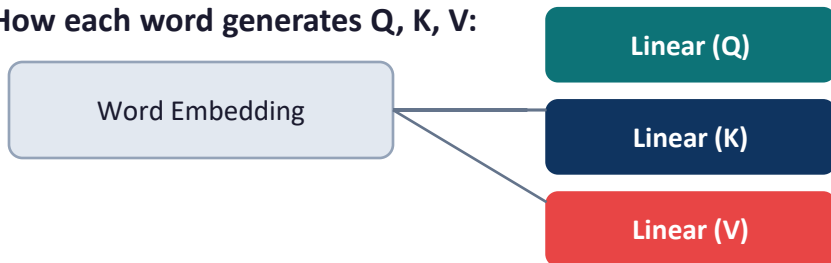
K
Key

What information do I contain?

V
Value

The actual information.

How each word generates Q, K, V:



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$$

1 Similarity (QK^T)

Compute dot product of Query with every Key.
Higher score = more relevant.

3 Softmax

Converts raw scores [4, 2, 1]
into probabilities [0.84, 0.11, 0.05] that sum to 1.

2 Scaling ($\div \sqrt{d_k}$)

Large vectors produce huge values $\rightarrow$ unstable softmax.
Dividing by $\sqrt{d_k}$ keeps gradients stable.

4 Weighted Sum ($\cdot V$)

Final output = $0.84 \cdot V_1 + 0.11 \cdot V_2 + 0.05 \cdot V_3$
Blends values by importance.

Example: $Q=[1,2] \cdot K_1=[2,2] = 6$ vs $Q=[1,2] \cdot K_2=[-1,3] = 5 \rightarrow K_1$ wins more attention

Numerical Example: "I love AI"

Step 1 — Assign Embeddings

I → [1, 0]
love → [0, 1]
AI → [1, 1]

Step 2 — Compute Q·K Scores (for word "I")

$Q \cdot K(I) = [1, 0] \cdot [1, 0] = 1$
 $Q \cdot K(\text{love}) = [1, 0] \cdot [0, 1] = 0$
 $Q \cdot K(\text{AI}) = [1, 0] \cdot [1, 1] = 1$

Step 3 — Apply Softmax ($\div \sqrt{2} \approx 1.41$)

Scaled: [0.71, 0, 0.71]
Softmax → [0.42, 0.16, 0.42]

Step 4 — Weighted Sum of Values

Output = $0.42 \cdot [1, 0] + 0.16 \cdot [0, 1] + 0.42 \cdot [1, 1]$
= [0.84, 0.58]

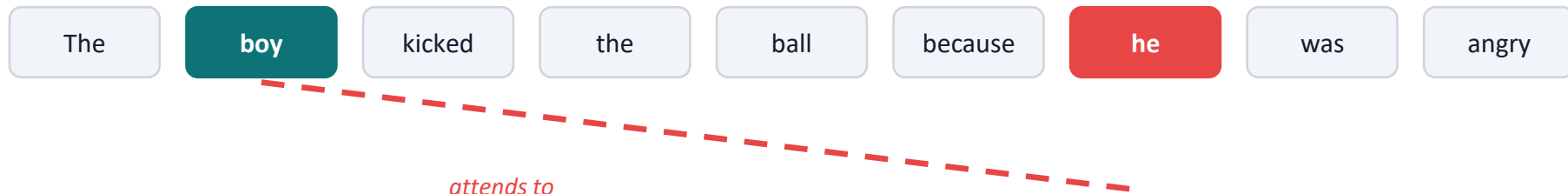
Task

Implement attention layer by code

Self-Attention

The boy kicked the ball because he was angry.

What does "he" refer to? → Self-attention lets each word attend to every other word.



Every word communicates with every other word.

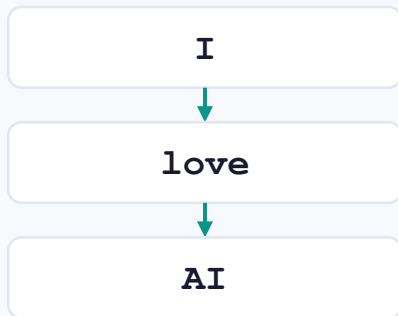
In self-attention, Q, K, and V all come from the SAME sequence.

Each word asks: "Which other words are most relevant to understanding me?"

The Attention Matrix

Attention scores for every word, against every other word — laid out as a grid.

SENTENCE



Each token will both ask a question (row) and be asked about (column).

Columns — who is being attended to?

Rows — who is asking?

	I	love	AI
I	0.6	0.2	0.2
love	0.2	0.5	0.3
AI	0.1	0.3	0.6

Each row sums to 1 — every word distributes its attention across the whole sentence.

Multi-Head Attention

Why only one attention? *Different heads learn different relationships.*



Head 1
Grammar



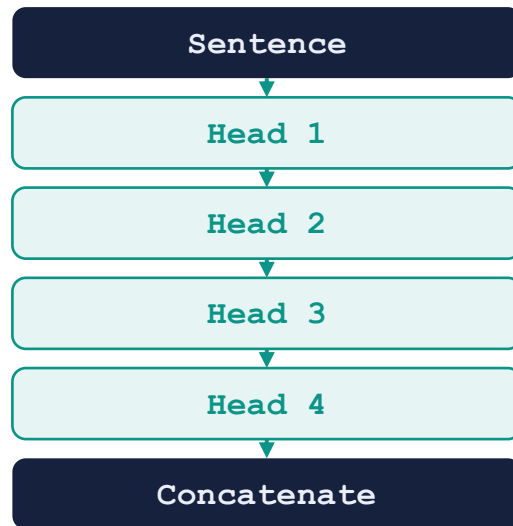
Head 2
Verb-object



Head 3
Pronouns



Head 4
Long-distance
dependency



Multiple heads run in parallel, each capturing a different kind of relationship — then their outputs are combined.

Positional Encoding

Attention ignores order — it only looks at content, not position.

WITHOUT POSITIONAL INFORMATION

Dog bites man

=

Man bites dog

As a bag of words, both sentences contain the exact same tokens — attention alone can't tell them apart.

We need to inject positional information — a signal for where each word sits in the sequence.

sin

Sine waves

Each position gets a unique wave pattern across the embedding dimensions — a fingerprint for position.

cos

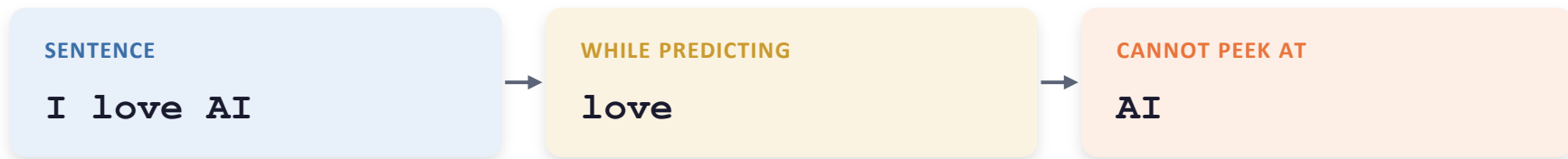
Cosine waves

Paired with sine at different frequencies, so every position maps to a distinct, smooth code.

These positional codes are added to word embeddings, so the model knows both what a word is and where it sits.

Masked Attention

Needed for GPT. A model predicting the next word can't be allowed to peek at it.



Future words are masked — set to $-\infty$ before softmax, so they receive zero attention weight.

	I	love	AI
I	✓	✗	✗
love	✓	✓	✗
AI	✓	✓	✓

Masking enforces left-to-right causality — each word can only attend to itself and what came before it.

Encoder vs. Decoder Attention

Three flavors of attention, depending on where they sit in the architecture.

ENCODER

Self-Attention

Everyone talks to everyone — full, unmasked attention across the whole input.

DECODER

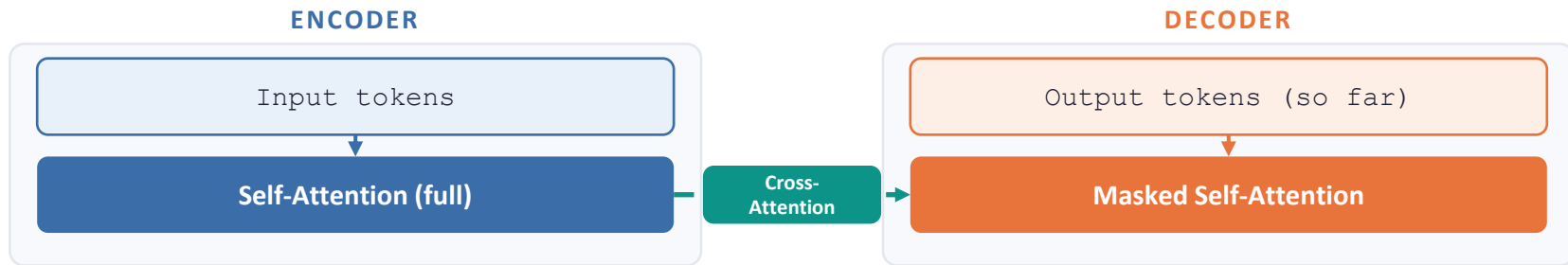
Masked Self-Attention

Cannot see future tokens — each position only attends to itself and the past.

ENCODER → DECODER

Cross-Attention

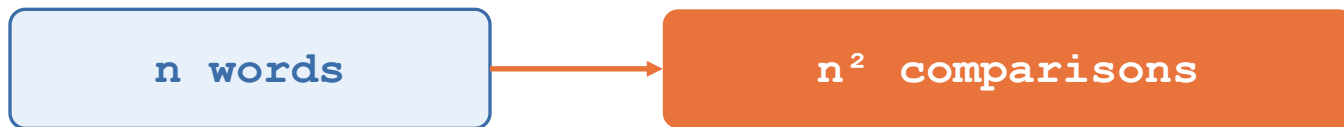
Decoder queries attend to the encoder's output representations.



Same attention mechanism, three jobs: full context (encoder), causal generation (decoder), and bridging the two (cross-attention).

The Cost of Attention

Every word compares itself to every other word — that comparison is what makes attention powerful, and expensive.



Every query attends to every key — so cost grows quadratically with sequence length.

EXAMPLE

1,000

tokens



1,000,000

pairwise comparisons

Efficient variants — **Linear Attention** and **FlashAttention** reduce this cost — through approximation or smarter memory access — to make long-context models practical.

Quadratic cost is the central scaling bottleneck of the Transformer — and a major area of ongoing research.

Attention in Large Language Models

The same building blocks from Parts 9–13, deployed inside the models you actually use.



GPT

Masked Self-Attention

Generates text one token at a time, attending only to itself and what came before — same causal mask from Part 12.



BERT

Bidirectional Self-Attention

Reads the whole sentence at once, with every token attending to every other token — built for understanding, not generation.



Encoder–Decoder

Self-Attention + Cross-Attention

Translation-style models use encoder self-attention, decoder masked self-attention, and cross-attention to connect them — the full architecture from Part 13.

Attention isn't one technique — it's a flexible mechanism reused across very different model families and goals.

Limitations of Attention

Attention is powerful, but not free — practical constraints to keep in mind.



Quadratic memory and computation

Cost scales as $O(n^2)$ with sequence length — the bottleneck explored in Part 14.



Degraded performance on long sequences

Even when compute allows it, quality can drop as context grows very long.



Attention weights aren't reliable explanations

High attention to a token doesn't always mean it drove the model's reasoning.



Motivates efficient attention research

Linear Attention, FlashAttention, and similar methods exist to address these limits.

Understanding these limits is what's driving the next generation of long-context, efficient Transformer designs.

Transformers:

Attention Is All You Need

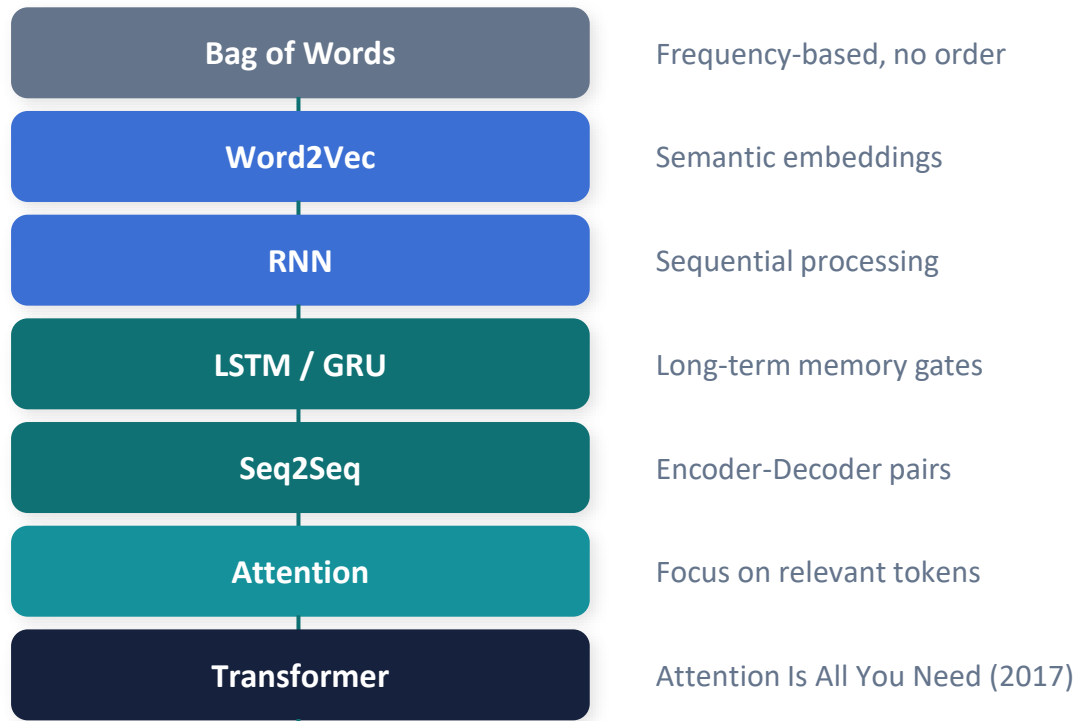
A Deep Dive into the Architecture that Changed AI

Part 1: Why Transformers?

Part 2: Architecture

Parts 3-7: Deep Dive

Evolution of NLP



Each step solved a limitation of the previous — Transformers solved them all.

Why Weren't RNNs Enough?

01

Sequential Computation

RNNs process one token at a time.
 $\text{Word}_1 \rightarrow \text{Word}_2 \rightarrow \text{Word}_3 \rightarrow \text{Word}_4$
Cannot be parallelised on modern GPUs.

02

Long-Term Dependencies

Example:
"The food that I ate yesterday at
the restaurant was amazing."

The model may forget the subject
by the time it reaches the end.

03

Slow Training

Processing one token at a time
makes training extremely slow.

Larger datasets = impossibly long
training times.



Which problem do you think was hardest to solve?

"Attention Is All You Need" — 2017



Attention Is All You Need

*Vaswani, Shazeer, Parmar
et al. (Google Brain, 2017)*

One of the most cited AI papers
of the 21st century.

Key Ideas



Remove recurrence completely

No more RNN/LSTM cells



Self-attention replaces sequencing

All tokens talk to all tokens at once



Fully parallelisable

GPUs can process all tokens simultaneously



Better long-range understanding

Equal access to any position

The Transformer removes recurrence entirely — relying solely on attention mechanisms.

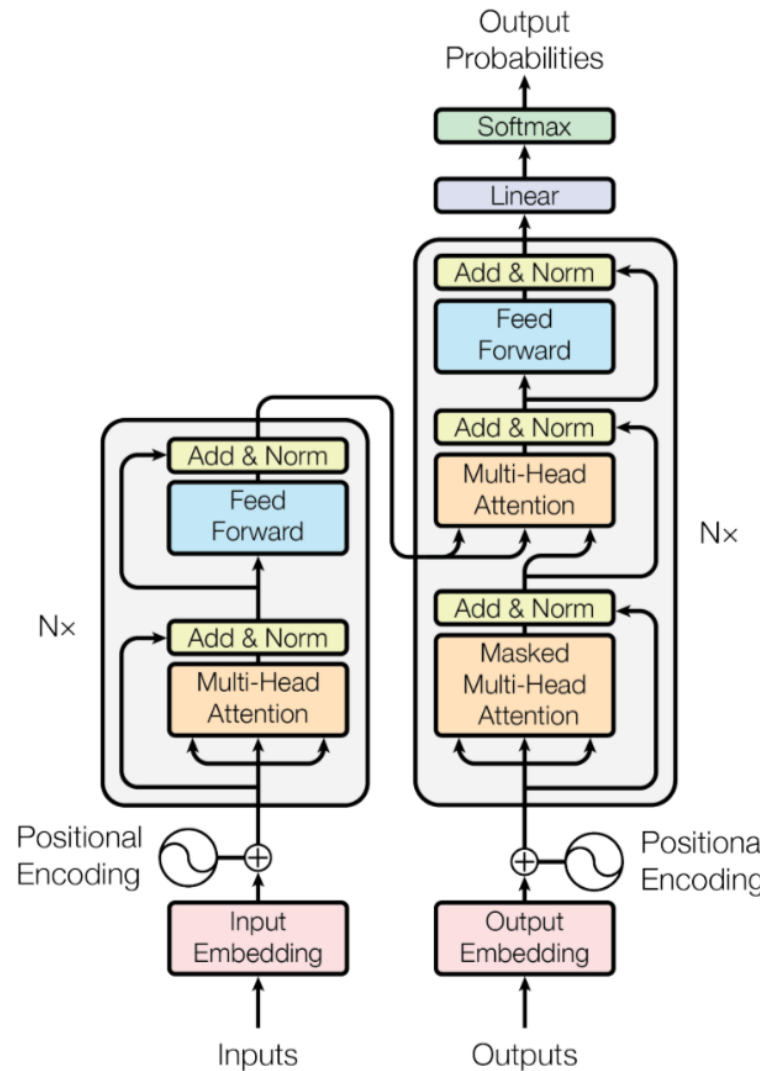
"Attention Is All You Need"



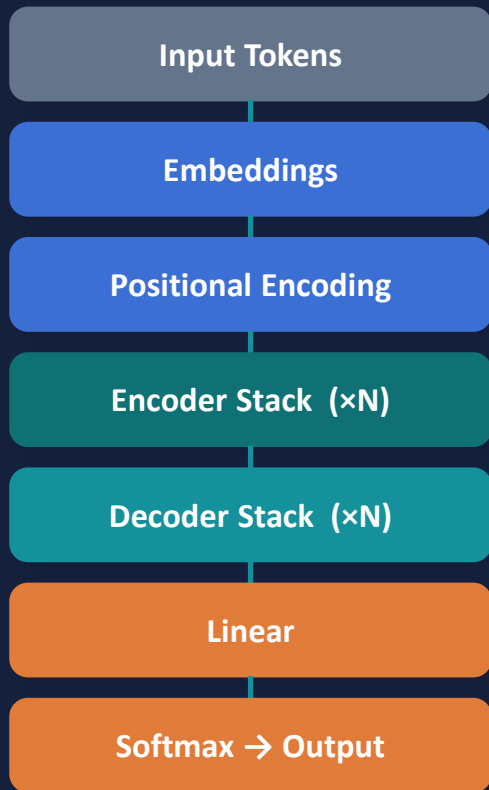
Attention Is All You Need

*Vaswani, Shazeer, Parmar
et al. (Google Brain, 2017)*

One of the most cited AI papers
of the 21st century.



Transformer: High-Level Architecture



Encoder

Processes the input sequence.
Builds rich contextual representations.

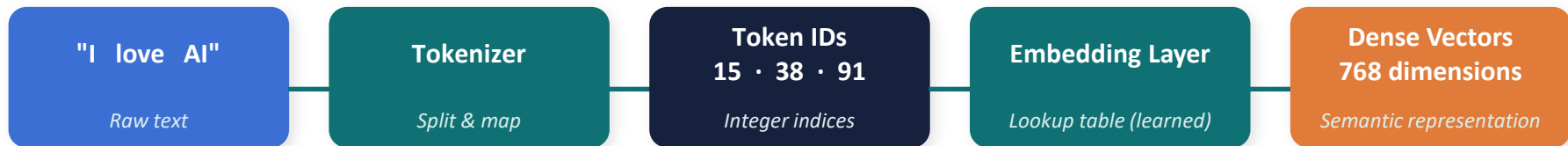
Decoder

Generates output tokens one by one.
Uses encoder context via cross-attention.

Output

Linear projection → vocabulary.
Softmax gives probability distribution.

Input Embeddings



Why embeddings?

Neural networks need numbers, not words. Embeddings map each token to a dense vector in a learned semantic space.

What does 768 mean?

Each word becomes a point in 768-dimensional space. Similar words are geometrically closer together.

Learned, not fixed

The embedding weights are updated during training. The model discovers which directions encode meaning.

Embeddings = learned dense vectors that transform discrete tokens into continuous semantic space.

Positional Encoding

⚠ The Problem

Attention ignores word order!

"Dog bites man" $\neq$ "Man bites dog"

...but without position info,
the embeddings look identical.



The Solution

Add position vectors to embeddings.

Each position gets a unique signature
based on sine & cosine functions:

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$$

Token
Embedding

+

Positional
Encoding

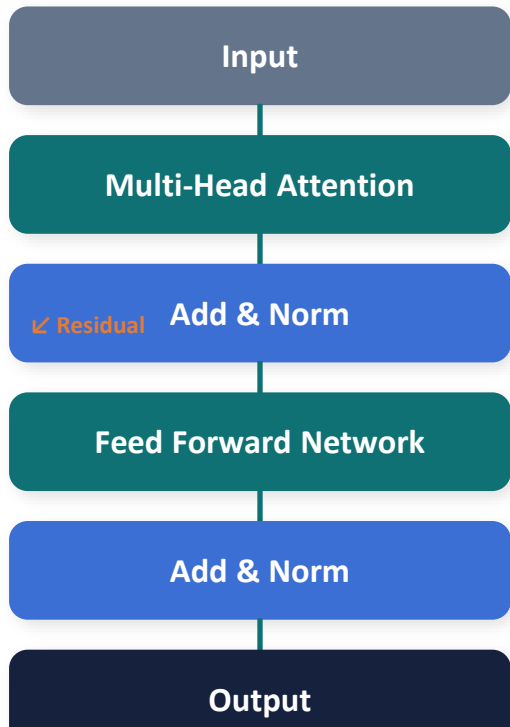
=

Final
Input Vector

Intuition: sine/cosine create unique, continuous patterns for every position that the model can learn to read.

Position is injected by adding a unique encoding vector to each token embedding before the encoder.

Encoder Block



Multi-Head Attention

Each token attends to all other tokens simultaneously. Multiple attention 'heads' learn different relationships (grammar, coreference, semantics).

Add & Norm (Residual + LayerNorm)

Output = Input + New_Info → prevents vanishing gradients and preserves original information.

Layer Norm keeps activations stable (like normalising exam scores).

Feed Forward Network

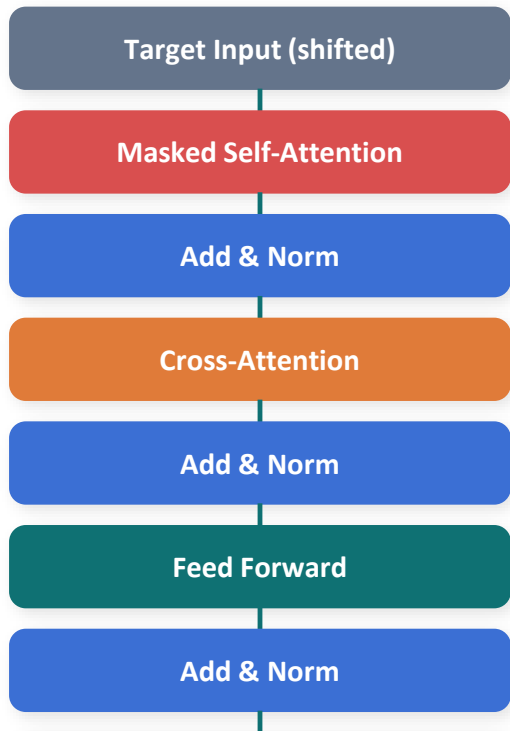
Simple 2-layer MLP: 768 → 3072 → 768

Applied independently to every token.

Adds non-linearity and capacity.

Each encoder block: attend → normalize → transform → normalize. Stack N of these for depth.

Decoder Block



Masked Self-Attention

Cannot look at future tokens during training.

To predict "love", the decoder can only see "I" — not "AI". This is the autoregressive mask.



Cross-Attention

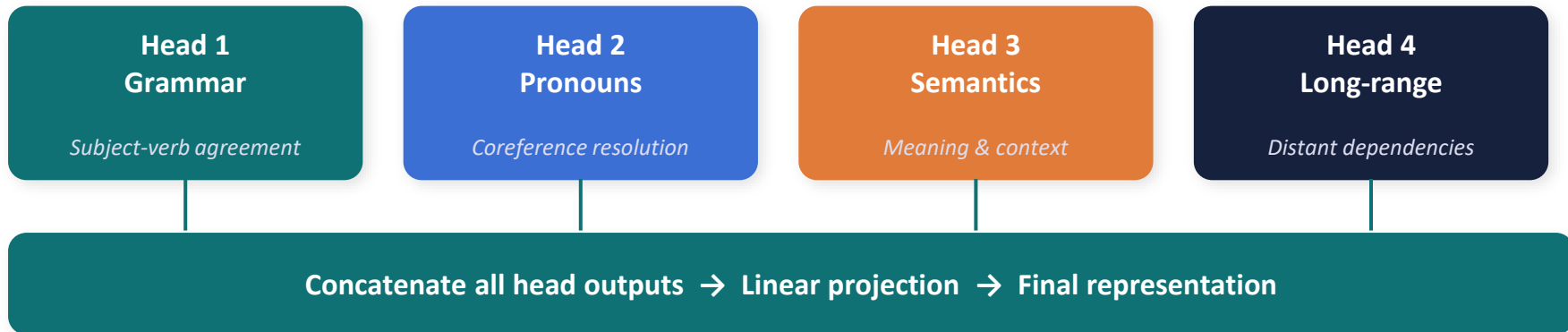
Decoder's Q comes from decoder; K, V come from encoder.

Example (translation):
Encoder sees "Bonjour" → Decoder generates "Hello"
Cross-attention connects the two sequences.

Decoder = masked self-attention + cross-attention (reads encoder) + feed-forward. Three sub-layers.

Multi-Head Attention

Why use multiple attention heads?



How attention works inside each head:

Query (Q) — what am I looking for? × Key (K) — what do I have? → Softmax → × Value (V) → Output

Multiple heads = multiple perspectives. Each head specialises in a different relationship type.

Training Process — Teacher Forcing

During training, the decoder is fed the correct previous tokens (ground truth) instead of its own predictions — this stabilizes and speeds up learning.

Example sentence: *"I love AI"*

Step 1

Input: I

Predict: love



Step 2

Input: I love

Predict: AI

Loss Computed



Backpropagation



*Weights
updated*

PART 9

At inference time, the model has no ground truth to rely on — it must generate text one token at a time, feeding each output back in as input.



Unlike training: tokens are generated one at a time, sequentially — not in parallel.

Encoder vs Decoder vs Encoder-Decoder

Model	Architecture	Tasks
Encoder Only	Encoder	Classification, NER, QA
Decoder Only	Decoder	Text Generation
Encoder-Decoder	Both	Translation, Summarization

Encoder

- BERT
- RoBERTa
- DeBERTa

Decoder

- GPT
- Llama
- Mistral
- Gemma

Encoder-Decoder

- T5
- BART
- FLAN-T5

Transformer Variants

BERT

- Bidirectional — reads the entire sentence at once.
- Great for understanding tasks.

GPT

- Autoregressive — predicts the next token.
- Great for generation tasks.

T5

- Everything becomes a text-to-text problem.
- Unifies tasks under one framework.

Llama

- Decoder-only Transformer.
- Optimized for large-scale LLMs.

Complexity — The Cost of Attention

$$O(n^2)$$

Attention Complexity

$$O(n^2)$$

Memory Usage

Why? Every token attends to every other token — doubling the sequence length quadruples the compute and memory required. This makes long-context processing expensive.

Mitigation techniques: *FlashAttention and sparse attention reduce the practical cost of long-context attention (implementation details are out of scope here).*

Transformer Applications



Machine Translation



Chatbots



Question Answering



Text Summarization



Code Generation



Speech Recognition



Image Captioning



Multimodal AI (text + images)

Transformer vs Previous Models

Feature	RNN	LSTM	Transformer
Parallel Training	No	No	Yes
Long Dependencies	No	Better	Yes
Attention	Limited	Limited	Core Mechanism
Training Speed	Slow	Slow	Fast
Scalability	Poor	Moderate	Excellent

The Transformer's parallelizable, attention-based design is what made today's large-scale LLMs possible.

BERT & GPT

From Transformer to Language Models

Encoder Only

Decoder Only

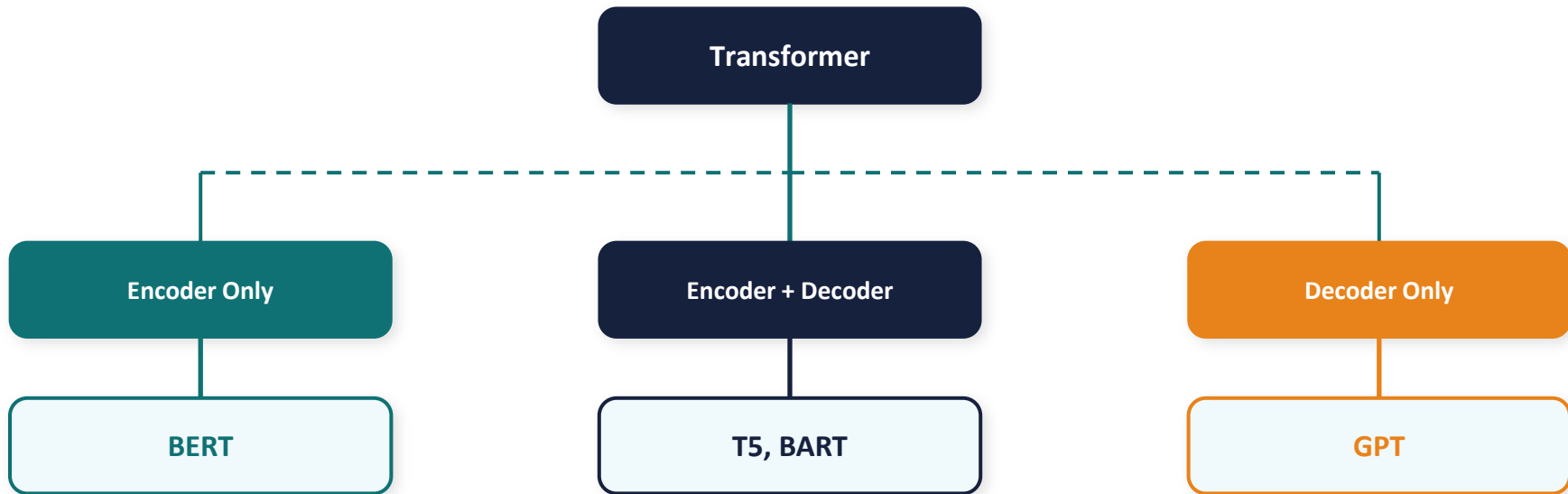
Bidirectional

Autoregressive

MLM

NSP

From Transformer to Language Models



BERT and GPT are specialized variants of the original Transformer architecture

What is BERT?

Bidirectional Encoder Representations from Transformers



Origin

Introduced by Google in 2018



Architecture

Uses only the Transformer Encoder



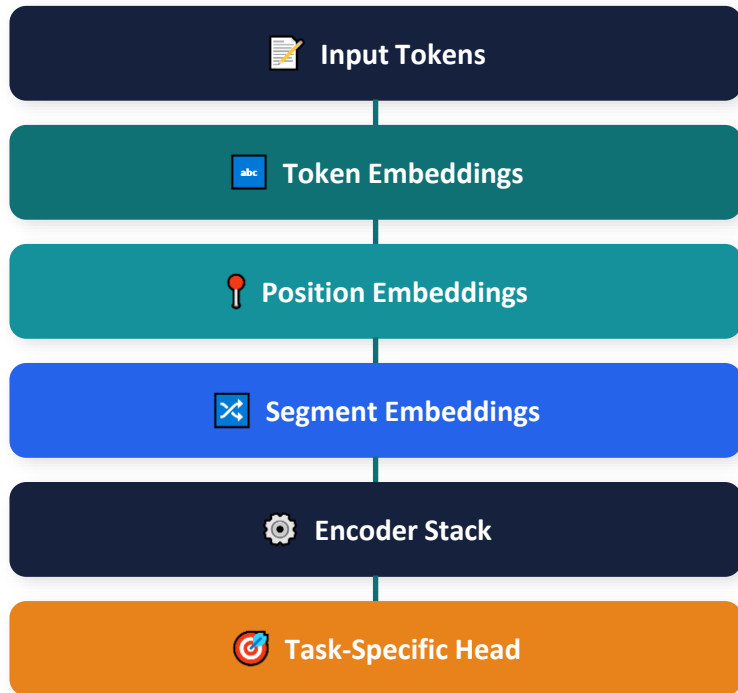
Reading

Reads the entire sentence simultaneously
— bidirectional

The **bank** is near the river .

← Both left and right context help BERT understand "bank" = riverbank, not a financial institution →

BERT Architecture

**[CLS]**

Classification token — always the first token. Used for sequence-level tasks.

[SEP]

Separator token — marks boundary between sentences A and B.

Token

WordPiece sub-word units. Unknown words split into pieces.

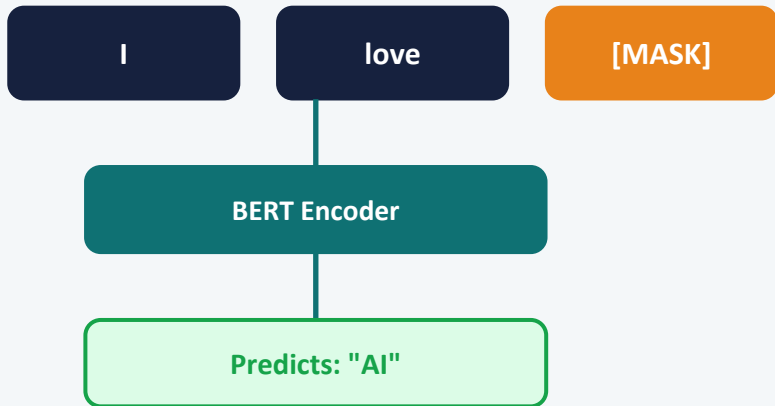
Seg A/B

Segment embeddings distinguish Sentence A from Sentence B.

[CLS] I love AI **[SEP]**

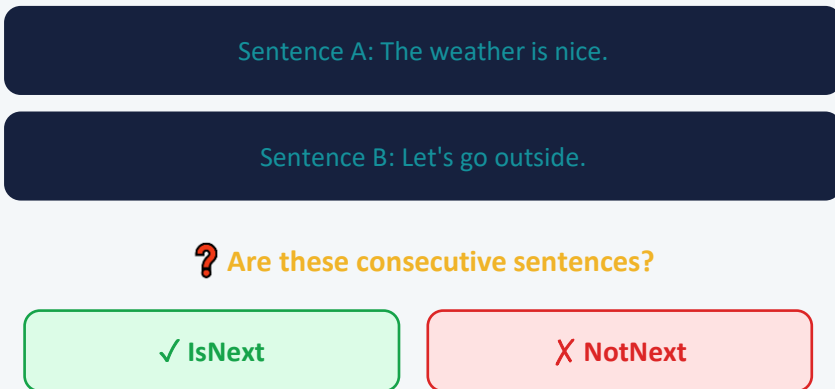
How BERT is Trained

① Masked Language Modeling (MLM)



15% of tokens are masked. BERT uses ALL surrounding words — left AND right — to predict the masked token.

② Next Sentence Prediction (NSP)



⚠ Many modern BERT variants (RoBERTa, etc.) dropped NSP — it proved less useful than MLM.

BERT Applications



Sentiment Analysis

Classify text as positive/negative/neutral



Text Classification

Assign categories to documents



Named Entity Recognition

Detect persons, orgs, locations



Question Answering

Find answers in passages (SQuAD)



Semantic Search

Match meaning, not just keywords



Document Classification

Sort long-form content by topic



BERT understands language — it excels at tasks requiring deep comprehension, not generation

What is GPT?

Generative Pre-trained Transformer



Origin

Introduced by OpenAI in 2018



Architecture

Uses only the Transformer Decoder



Generation

Generates text one token at a time —
left-to-right

Generation Example

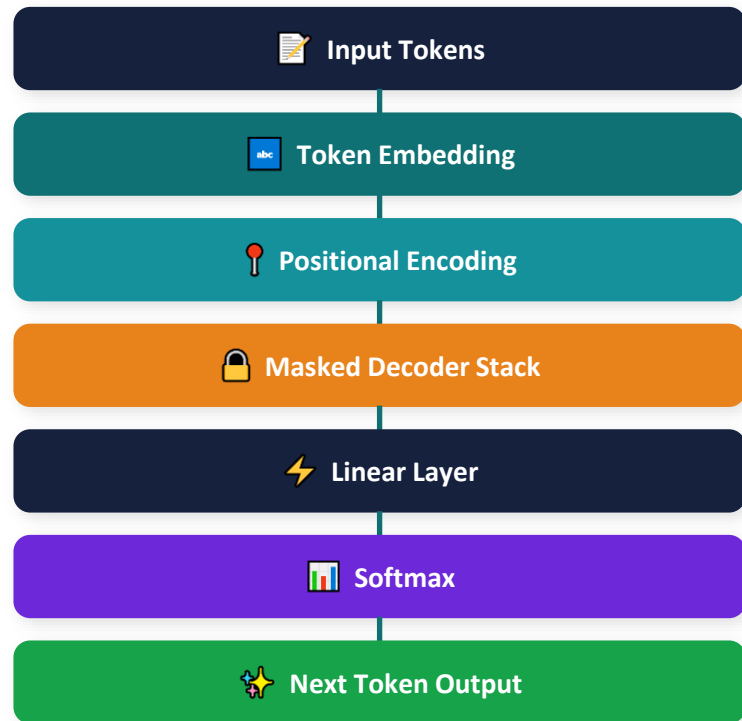
Input: "Once upon a time,"



"...there was a brave knight who fought a dragon..."

GPT generates language — it predicts the next token autoregressively, left-to-right

GPT Architecture



Masked Self-Attention

GPT can only attend to past and current tokens — future tokens are masked (hidden).



Visible to GPT ✓

Hidden X

Key Difference vs BERT

BERT → Bidirectional. Sees ALL tokens simultaneously.

GPT → Unidirectional. Sees only PAST tokens.

GPT's masked decoder means it can only look left — this enables natural text generation

How GPT is Trained

Training Objective: Next Token Prediction (Autoregressive Language Modeling)

Step 1

Input: "I"

Predict: "love"

Correct? Update weights.

Step 2

Input: "I love"

Predict: "AI"

Correct? Update weights.

Step 3

Input: "I love AI"

Predict: "."

Correct? Update weights.

Autoregressive: each output token is fed back as input to predict the next token — the model builds text step by step.

GPT Applications



Chatbots

Conversational AI assistants



Text Generation

Articles, essays, creative writing



Story Writing

Narrative and creative fiction



Code Generation

Write and debug code snippets



Translation

Cross-lingual text transfer



Summarization

Condense long documents



Question Answering

Generative QA from knowledge



Email Writing

Draft professional messages



GPT generates language — it excels at any task requiring fluent, coherent text creation

BERT vs GPT

The Most Important Comparison

Feature	 BERT	 GPT
Transformer Part	Encoder	Decoder
Reading Direction	Bidirectional $\leftrightarrow$	Left-to-right $\rightarrow$
Training Objective	Masked Word Prediction	Next Token Prediction
Main Purpose	Understanding	Generation
Attention Type	Full Self-Attention	Masked Self-Attention
Best Tasks	Classification, QA, NER	Chatbots, Writing, Code
BERT  Understands Language GPT  Generates Language		